

IMPLEMENTASI ALGORITMA ADVANCED ENCRYPTION STANDARD (AES) UNTUK PENGAMANAN FILE MENGGUNAKAN BAHASA PEMROGRAMAN PYTHON

Penulis

Nama : Rohmani Firdaus

Program Studi : D3 TEKNIK INFORMATIKA

Abstrak

Perkembangan teknologi informasi menyebabkan pertukaran data digital menjadi semakin cepat. Namun, kemudahan tersebut juga meningkatkan risiko kebocoran informasi akibat akses yang tidak sah. Salah satu solusi yang dapat diterapkan adalah penggunaan teknik kriptografi. Penelitian ini bertujuan mengimplementasikan algoritma **Advanced Encryption Standard (AES)** berbasis Python untuk melakukan enkripsi dan dekripsi file. Sistem dibangun menggunakan pustaka **PyCryptodome**, dengan algoritma **AES mode CBC (Cipher Block Chaining)** serta pembentukan kunci menggunakan fungsi hash **SHA-256** berdasarkan password pengguna. Hasil implementasi menunjukkan bahwa file berhasil diubah menjadi ciphertext sehingga tidak dapat dibaca tanpa proses dekripsi menggunakan password yang benar. Pengujian menunjukkan bahwa proses enkripsi dan dekripsi berjalan dengan baik pada berbagai jenis file.

Kata kunci: Kriptografi, AES, Python, SHA-256, Keamanan Data.

1. Pendahuluan

Perkembangan internet menyebabkan pertukaran informasi digital meningkat secara signifikan. Data yang dikirimkan melalui jaringan komputer berpotensi mengalami penyadapan maupun manipulasi apabila tidak diberikan mekanisme keamanan yang memadai. Kriptografi merupakan ilmu yang digunakan untuk menjaga kerahasiaan informasi dengan mengubah data asli (*plaintext*) menjadi data yang tidak dapat dipahami (*ciphertext*). Salah satu algoritma kriptografi modern yang banyak digunakan adalah **Advanced Encryption Standard (AES)** karena memiliki tingkat keamanan yang tinggi serta performa yang cepat. AES telah ditetapkan sebagai standar enkripsi oleh National Institute of Standards and Technology (NIST).

Penelitian ini mengimplementasikan algoritma AES menggunakan bahasa pemrograman Python sehingga pengguna dapat mengenkripsi maupun mendekripsi berbagai jenis file secara sederhana.

2. Rumusan Masalah

Rumusan masalah penelitian ini adalah:

1. Bagaimana mengimplementasikan algoritma AES pada aplikasi Python?
2. Bagaimana proses enkripsi dan dekripsi file menggunakan password?
3. Apakah sistem mampu menjaga kerahasiaan isi file dari akses yang tidak sah?

3. Tujuan Penelitian

Tujuan penelitian ini adalah:

- Mengimplementasikan algoritma AES pada aplikasi berbasis Python.
- Mengembangkan aplikasi enkripsi dan dekripsi file.
- Menguji keberhasilan proses enkripsi dan dekripsi menggunakan password.

4. Landasan Teori

4.1 Kriptografi

Kriptografi merupakan teknik pengamanan informasi melalui proses matematis sehingga hanya pihak yang memiliki kunci yang dapat membaca informasi tersebut.

Tujuan utama kriptografi meliputi:

- Confidentiality
- Integrity
- Authentication
- Non-repudiation

4.2 Advanced Encryption Standard (AES)

AES merupakan algoritma kriptografi simetris yang menggunakan satu kunci yang sama untuk proses enkripsi maupun dekripsi.

Karakteristik AES:

- Block Size : 128 bit
- Key Size : 128, 192, atau 256 bit
- Cepat
- Aman
- Digunakan secara luas

AES bekerja melalui beberapa tahapan transformasi, yaitu:

1. SubBytes

2. ShiftRows
3. MixColumns
4. AddRoundKey

Standar AES didefinisikan oleh NIST dalam FIPS PUB 197.

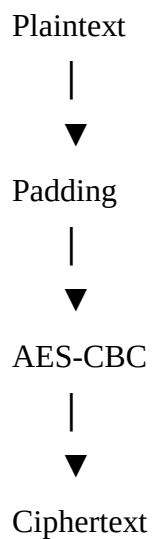
4.3 Mode CBC (Cipher Block Chaining)

Program menggunakan mode CBC.

Keunggulan CBC:

- Menggunakan Initialization Vector (IV)
- Ciphertext setiap blok berbeda
- Lebih aman dibanding ECB

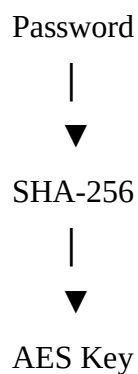
Pada implementasi ini:



4.4 SHA-256

Password pengguna tidak digunakan secara langsung sebagai kunci AES.

Password terlebih dahulu diproses menggunakan algoritma SHA-256 sehingga menghasilkan kunci sepanjang 256 bit.



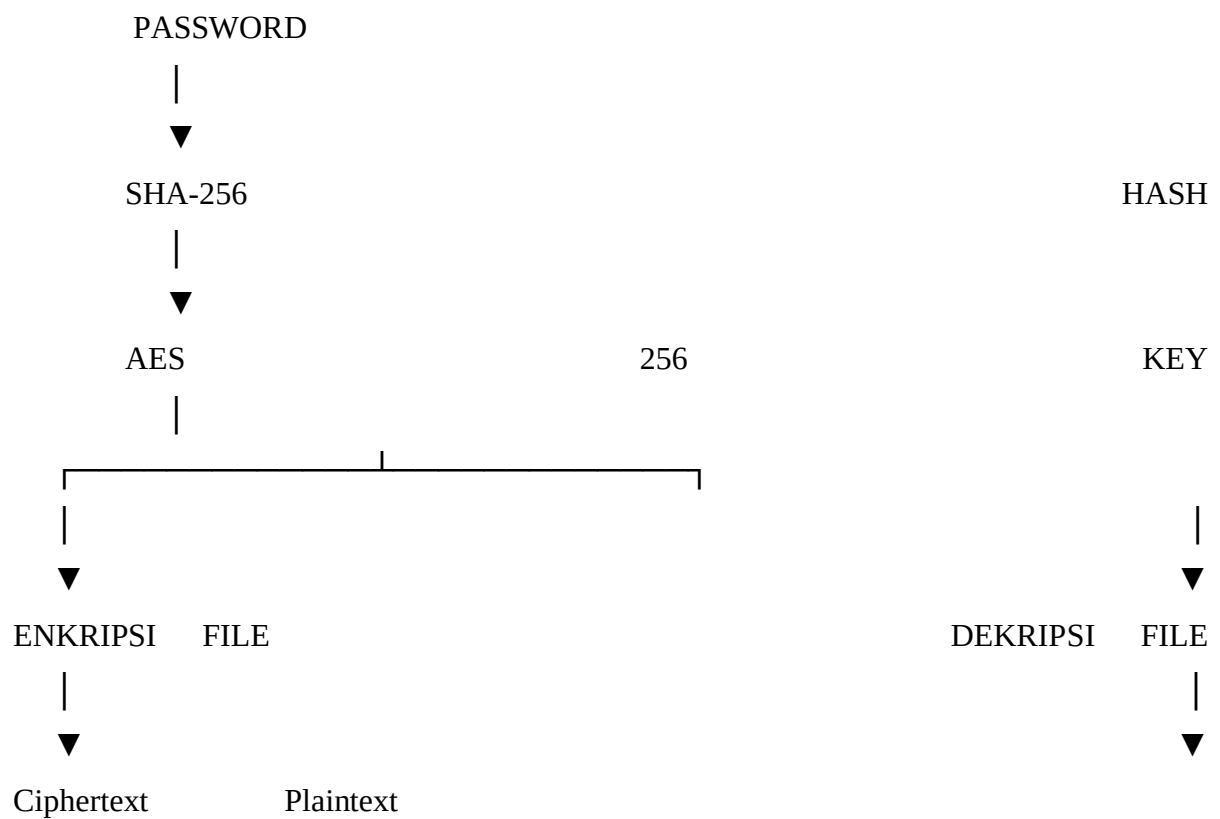
5. Metode Penelitian

Metode penelitian yang digunakan terdiri dari:

1. Studi Literatur
2. Perancangan Sistem
3. Implementasi Program
4. Pengujian Sistem
5. Analisis Hasil

6. Perancangan Sistem

Diagram proses sistem:



7. Implementasi Program

Program dikembangkan menggunakan:

- Python 3
- PyCryptodome
- hashlib

Fungsi utama terdiri dari:

get_key()

Mengubah password menjadi AES Key.

def

 return hashlib.sha256(password.encode()).digest()

encrypt_file()

Melakukan:

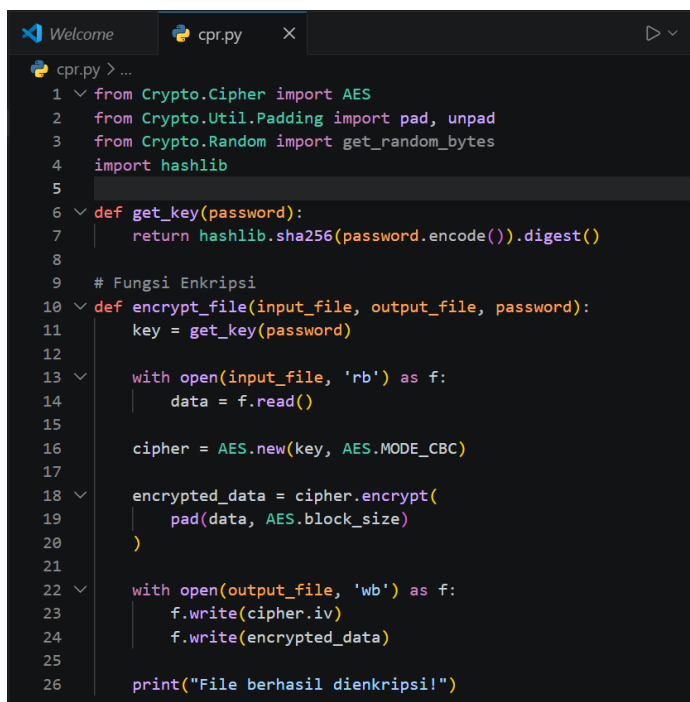
- membaca file
- padding
- AES CBC
- menyimpan IV
- menyimpan ciphertext

decrypt_file()

Melakukan:

- membaca IV
- membaca ciphertext
- AES decrypt
- unpadding
- menyimpan hasil

get_key(password):



```
1 from Crypto.Cipher import AES
2 from Crypto.Util.Padding import pad, unpad
3 from Crypto.Random import get_random_bytes
4 import hashlib
5
6 def get_key(password):
7     return hashlib.sha256(password.encode()).digest()
8
9 # Fungsi Enkripsi
10 def encrypt_file(input_file, output_file, password):
11     key = get_key(password)
12
13     with open(input_file, 'rb') as f:
14         data = f.read()
15
16     cipher = AES.new(key, AES.MODE_CBC)
17
18     encrypted_data = cipher.encrypt(
19         pad(data, AES.block_size)
20     )
21
22     with open(output_file, 'wb') as f:
23         f.write(cipher.iv)
24         f.write(encrypted_data)
25
26     print("File berhasil dienkripsi!")
27
```

```

cpr.py > ...
27
28 def decrypt_file(input_file, output_file, password):
29     key = get_key(password)
30
31     with open(input_file, 'rb') as f:
32         iv = f.read(16)
33         encrypted_data = f.read()
34
35     cipher = AES.new(key, AES.MODE_CBC, iv)
36
37     decrypted_data = unpad(
38         cipher.decrypt(encrypted_data),
39         AES.block_size
40     )
41
42     with open(output_file, 'wb') as f:
43         f.write(decrypted_data)
44
45     print("File berhasil didekripsi!")
46
47 # Menu
48 print("=== AES FILE ENCRYPTION ===")
49 print("1. Enkripsi")
50 print("2. Dekripsi")
51
52 pilihan = input("Pilih menu : ")
53
54 if pilihan == "1":
55     infile = input("Nama file input : ")
56     outfile = input("Nama file output : ")
57     password = input("Password : ")
58
59     encrypt_file(infile, outfile, password)
60
61 elif pilihan == "2":
62     infile = input("Nama file terenkripsi : ")
63     outfile = input("Nama file hasil dekripsi : ")
64     password = input("Password : ")
65
66     decrypt_file(infile, outfile, password)
67
68 else:
69     print("Pilihan tidak tersedia")

```

8. Hasil Pengujian

Pengujian dilakukan menggunakan file Python.

Input:

File :

cpr.py

Password :

2402072

Output:

File berhasil dienkrpsi!

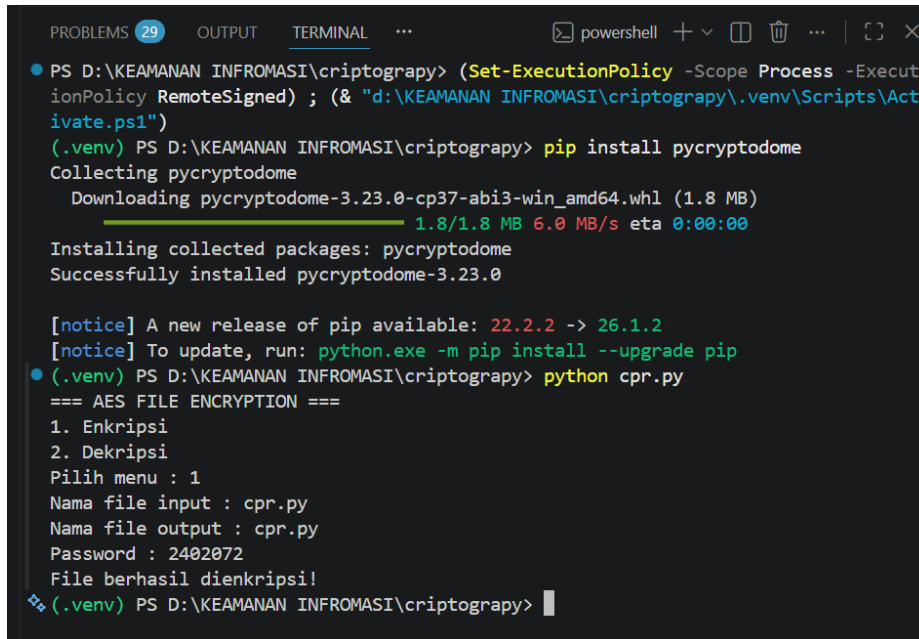
Hasil file berubah menjadi data biner yang tidak dapat dibaca menggunakan text editor.

Selanjutnya dilakukan proses dekripsi menggunakan password yang sama.

Output:

File berhasil didekripsi!

Isi file berhasil kembali seperti semula.



```
PS D:\KEAMANAN INFROMASI\criptograby> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& "d:\KEAMANAN INFROMASI\criptograby\.venv\Scripts\Activate.ps1")
(.venv) PS D:\KEAMANAN INFROMASI\criptograby> pip install pycryptodome
Collecting pycryptodome
  Downloading pycryptodome-3.23.0-cp37-abi3-win_amd64.whl (1.8 MB)
    1.8/1.8 MB 6.0 MB/s eta 0:00:00
Installing collected packages: pycryptodome
Successfully installed pycryptodome-3.23.0

[notice] A new release of pip available: 22.2.2 -> 26.1.2
[notice] To update, run: python.exe -m pip install --upgrade pip
(.venv) PS D:\KEAMANAN INFROMASI\criptograby> python cpr.py
=== AES FILE ENCRYPTION ===
1. Enkripsi
2. Dekripsi
Pilih menu : 1
Nama file input : cpr.py
Nama file output : cpr.py
Password : 2402072
File berhasil didekripsi!
(.venv) PS D:\KEAMANAN INFROMASI\criptograby>
```

9. Analisis

Implementasi menunjukkan bahwa:

- Password diubah menjadi kunci AES menggunakan SHA-256.
- AES berhasil mengenkripsi seluruh isi file.
- IV disimpan pada 16 byte pertama file hasil enkripsi.
- Password yang salah akan menyebabkan proses dekripsi gagal (misalnya karena kesalahan padding), sehingga isi file tidak dapat dipulihkan.

Walaupun implementasi ini bekerja dengan baik untuk pembelajaran, penggunaan SHA-256 secara langsung sebagai pembentuk kunci masih memiliki kelemahan terhadap serangan *brute force* apabila password lemah. Praktik yang lebih direkomendasikan adalah menggunakan *password-based key derivation function* seperti PBKDF2, scrypt, atau Argon2, serta mode enkripsi terautentikasi seperti AES-GCM.

10. Kelebihan Sistem

- Mudah digunakan.
- Menggunakan AES yang merupakan standar internasional.
- Dapat mengenkripsi berbagai jenis file.

- Program sederhana dan mudah dikembangkan.

11. Kekurangan Sistem

- Belum menggunakan GUI.
- Password belum menggunakan salt.
- Belum menggunakan PBKDF2.
- Belum ada autentikasi integritas (MAC atau AES-GCM).
- Belum mendukung enkripsi folder.

12. Kesimpulan

Penelitian ini berhasil mengimplementasikan algoritma AES berbasis Python untuk melakukan enkripsi dan dekripsi file. Penggunaan AES-CBC dengan kunci yang dibentuk melalui SHA-256 mampu menjaga kerahasiaan data sehingga file tidak dapat dibaca tanpa password yang benar. Program berhasil mengenkripsi serta mengembalikan isi file ke bentuk semula melalui proses dekripsi. Untuk pengembangan selanjutnya, disarankan menggunakan PBKDF2 atau Argon2 sebagai pembentuk kunci serta mode AES-GCM agar memperoleh jaminan kerahasiaan sekaligus integritas data.

Daftar Pustaka

1. National Institute of Standards and Technology. **Advanced Encryption Standard (AES), FIPS PUB 197**. 2023.
2. Herman, H., Wijaya, R., Miharja, S., & Wilson. *Implementasi Algoritma AES-128 dan SHA-256 dalam Perancangan Aplikasi Pengamanan File Dokumen*. Jurnal TIMES, 2022.
3. Utama, F. P., Wijaya, G., Faurina, R., & Vatesia, A. *Implementasi Algoritma AES 256 CBC, BASE64, dan SHA-256 dalam Pengamanan dan Validasi Data Ujian Online*. JTIK, 2023.
4. Rosandi, A. A., dkk. *Design of File Encryption and Decryption Application Using AES-256 Cryptography Algorithm Based on Streamlit*. International Journal of Multidisciplinary Research and Literature, 2025.
5. Latip, P. N. *Implementasi Algoritma Kriptografi AES dalam Pengamanan File Teks*. Jurnal Riset Sistem Informasi, 2025.